

Exercice 01 : (6pts)**1) Solution avec les sémaphores (3 pts)**

```
#define M 10;
```

```
semaphore mutex = 1, c = 0; int nbc = 0; // Nombre de carrosseries
```

```
Processus Carrosserie_i ( ) { // (i = 1..N)
```

```
    wait (mutex)
```

```
    <Déposer la carrosserie>
```

```
    nbc := nbc + 1
```

```
    if nbc = M alors signal (c)
```

```
        else signal (mutex)
```

```
    }
```

```
Processus Peinture ( ) {
```

```
    while (1){
```

```
        wait (c)
```

```
        < Peindre toutes les carrosseries>
```

```
        nbc := 0
```

```
        signal (mutex)
```

```
    }
```

```
}
```

2) Solution avec le moniteur (3 pts)

```
Monitor Déposer _ Peintre {
```

```
    Condition DéposerOK , PeintreOK ;
```

```
    int nbc = 0 ;
```

```
void Déposer ( ) {
```

```
    if (nbc == M) cwait (DéposerOK);
```

```
    < Déposer la carrosserie >
```

```
    nbc := nbc + 1
```

```
    if (nbc == M) csignal (PeintreOK);
```

```
}
```

```
void Peintre ( ) {
```

```
    if (nbc < M) cwait (PeintreOK)
```

```
    < Peindre toutes les carrosseries >
```

```
    nbc := 0
```

```
    csignal (DéposerOK);
```

```
}
```

```
{ // code d'initialisation
```

```
    nbc = 0 ; } // fin moniteur
```

```
Processus Carrosserie_i ( ) {
```

```
    Déposer _ Peintre.Déposer ( )
```

```
}
```

```
Processus Peinture ( ) {
```

```
    while(1){
```

```
        Déposer _ Peintre. Peintre ( )
```

```
    }
```

```
}
```

```
int main ( ) {
```

```
    parbegin(Painture, Carrosserie_1, Carrosserie_2, ..., Carrosserie_N);
```

```
    return 0 ;
```

```
}
```

Exercice 02 : (8pts)

Q1- Les deux approches proposées pour résoudre le problème d'attente circulaire afin d'éviter l'interblocage. (2pts)

- **1^{ère} approche** : Limiter le nombre de ressources détenues par un processus à un instant donné, s'il a besoin d'une nouvelle, il faut libérer une autre.
- **2^{ème} approche** : Numéroté toutes les ressources d'une manière croissante, si un processus demande une ressource de numéro supérieur de ce qu'il détient on lui donne cette ressource, sinon si le nombre est inférieur on refuse sa demande → il faut demander une ressource avec un numéro supérieur.

Q2- La différence entre les deux sémaphores binaire et générale (Dijkstra) est que : le processus qui verrouille un mutex (met sa valeur à zéro) est le seul à pouvoir le déverrouiller (met sa valeur à 1). Par contre, il est possible qu'un sémaphore binaire soit verrouillé par un processus et déverrouiller par un autre. (2pts)

Q3- Faux (1pt)

Q4- Les étapes d'exécution de l'algorithme de Parcours en Profondeur (DFS). (2pts)

1. Initialiser **L** une pile vide et designer tous les arcs comme non marqué
2. Pour chaque nœud **n** :
3. Si le nœud **n** n'est pas dans la pile **L**, l'ajouter à **L**
Sinon, il existe un cycle et l'algorithme prend fin
4. Si il n'existe pas un arc non marqué sortant de **n**, dépiler **n**
Si la pile est vide, aller à **l'étape 2**
Sinon le nœud au sommet de la pile devient **n** et revenir à **l'étape 4**
Sinon choisir au hasard un arc sortant non marqué et marquer le, pointer sur le nœud au bout de l'arc choisi qui devient **n** et revenir à **l'étape 3**
Si l'algorithme ne se termine pas à l'étape 3, il n'y a pas de cycle (pas d'interblocage)

Q5- Vrai (1pts)

Exercice 03 :(6pts)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
sem_t a,b,c;

void* mafonction1(void* arg){
    sem_wait(&a);
    printf("X");
    sem_post(&b);
    sem_wait(&c);
    printf("?");
}

void* mafonction2(void* arg){
    printf("Y");
    sem_post(&a);
}

void* mafonction3(void* arg){
    sem_wait(&b);
    printf("Z");
    sem_post(&c);
}

int main() {
    pthread_t p1, p2,p3;

    sem_init(&a,0,0);
    sem_init(&b,0,0);
    sem_init(&c,0,0);
    pthread_create(&p1, NULL,&mafonction1, NULL);
    pthread_create(&p2, NULL,&mafonction2, NULL);
    pthread_create(&p3, NULL,&mafonction3, NULL);

    getchar();
    return 0;
}
```